

Design Analysis & Algorithm - Assignment 4

Q.1)

The Bellman-Ford algorithm normally relaxes all $|V|-1$ times, but we don't always need to do that. After each full pass over all edges, if none of the distances change, then all the shortest paths have already been found, because any improvement would have shown up during that pass. So we will keep a flag during every iteration & see it whenever a relaxation updates a vertex's distance. If after an entire pass the flag is still FALSE, that means no distance can be improved further, and then we can stop the algorithm early instead of completing all remaining passes. This works ~~as~~ ^{because} once no edge can reduce its endpoint's distance, every reachable vertex already has the correct shortest-path value. We will still do one final check to see if any edge can be relaxed again; if it can then that indicates a negative weight cycle. Otherwise, the algorithm finishes with the correct shortest paths.

BELLMAN FORD EARLY TERMINATION (G, w, s)

//initialization

for each vertex v in $V[G]$ $v.d = \infty$ $v.\pi = \text{nil}$ $s.d = 0$ //relax edges repeatedly until no update happens
repeat

changed = FALSE

for each edge (u, v) in $E[G]$ if $v.d > u.d + w(u, v)$ $v.d = u.d + w(u, v)$ $v.\pi = u$

changed = TRUE

until changed == FALSE

KAGHAZ
www.kaghazpk

// check for negative-weight cycles
 for each edge (u, v) in $E[G]$
 if $v.d > u.d + w(u, v)$
 return "negative weight cycle detected"

return distances and predecessors

Q.2) $r \rightarrow s \rightarrow t \rightarrow x \rightarrow y \rightarrow z$

$d[r] = 0, \pi[r] = \text{nil}$

$d[s] = \infty, \pi[s] = \text{nil}$

$d[t] = \infty, \pi[t] = \text{nil}$

$d[x] = \infty, \pi[x] = \text{nil}$

$d[y] = \infty, \pi[y] = \text{nil}$

$d[z] = \infty, \pi[z] = \text{nil}$

vertex	edge	updated $d[]$ values	updated $\pi[]$ values
r	$r \rightarrow s(5)$	$s=5, d[s]=5$	$\pi[s]=r$
	$r \rightarrow t(3)$	$t=3, d[t]=3$	$\pi[t]=r$
		$d[r]=0, d[x]=d[y]$ $d[z]=\infty$	
s	$s \rightarrow t(2)$	$3 < 5+2, d[t]=3$	
	$s \rightarrow x(6)$	$x=5+6=11, d[x]=11$	$\pi[x]=s$
t	$t \rightarrow x(7)$	$x=3+7=10, d[x]=10$	$\pi[x]=t$
	$t \rightarrow y(4)$	$y=3+4=7, d[y]=7$	$\pi[y]=t$
	$t \rightarrow z(2)$	$z=3+2=5, d[z]=5$	$\pi[z]=t$
x	$x \rightarrow y(-1)$	$7 < 10+(-1), d[y]=7$	
	$x \rightarrow z(1)$	$5 < 10+1, d[z]=5$	
y	$y \rightarrow z(-2)$	$5 = 7+(-2), d[z]=5$	

$\hookrightarrow$ time complexity: $O(V+E)$

$\hookrightarrow$ final path:

$r \rightarrow t \rightarrow z$

$\hookrightarrow$ cost = 5



KAGHAZ
www.kaghaz.pk

Q.3)

↳ 's' as source vertex

vertex	D[s]	D[t]	D[x]	D[y]	D[z]	$\pi[t]$	$\pi[x]$	$\pi[y]$	$\pi[z]$	S
s	0	10, s	∞	<u>5, s</u>	∞	s	-	s	-	{s}
y	0	8, y	14, y	<u>7, y</u>		y	y	s	y	{s, y}
z	0	<u>8, y</u>	13, z			y	z	s	y	{s, y, z}
t	0		<u>9, t</u>			y	t	s	y	{s, y, z, t}
x	0	8	9	5	z	y	t	s	y	{s, y, z, t, x}

↳ 'z' as source vertex

vertex	D[z]	D[y]	D[x]	D[t]	D[s]	$\pi[y]$	$\pi[x]$	$\pi[t]$	$\pi[s]$	S
z	0	∞	<u>6, z</u>	∞	7, z	-	z	-	z	{z}
x	0	∞		∞	<u>7, z</u>	-	z	-	z	{z, x}
s	0	<u>12, s</u>		17, s		s	z	s	z	{z, x, s}
y	0			<u>15, y</u>		s	z	y	z	{z, x, s, y}
t	0	12	6	15	7	s	z	y	z	{z, x, s, y, t}

$$Q.4) A_k[i, j] = \min[A^{k-1}[i, j], A^{k-1}[i, k] + A^{k-1}[k, j]]$$

$$a) A_1 = \begin{bmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ 1 & 0 & \infty & 2 & \infty & \infty \\ \infty & 2 & 0 & \infty & \infty & -8 \\ -4 & \infty & \infty & 0 & 3 & \infty \\ \infty & 7 & \infty & \infty & 0 & \infty \\ \infty & 5 & 10 & \infty & \infty & 0 \end{bmatrix} \quad A_2 = \begin{bmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ 1 & 0 & \infty & 2 & 0 & \infty \\ \infty & 2 & 0 & \infty & \infty & -8 \\ -4 & \infty & \infty & 0 & -5 & \infty \\ \infty & 7 & \infty & \infty & 0 & \infty \\ \infty & 5 & 10 & \infty & \infty & 0 \end{bmatrix}$$

$$A_2 = \begin{bmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ 1 & 0 & \infty & 2 & 0 & \infty \\ 3 & 2 & 0 & 4 & 2 & -8 \\ -4 & \infty & \infty & 0 & -5 & \infty \\ 8 & 7 & \infty & 9 & 0 & \infty \\ 6 & 5 & 10 & 7 & 5 & 0 \end{bmatrix} \quad A_3 = \begin{bmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ 1 & 0 & \infty & 2 & 0 & \infty \\ 3 & 2 & 0 & 4 & 2 & -8 \\ -4 & \infty & \infty & 0 & -5 & \infty \\ 8 & 7 & \infty & 9 & 0 & \infty \\ 6 & 5 & 10 & 7 & 5 & 0 \end{bmatrix}$$

day / date:

$$A^4 = \begin{bmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ -2 & 0 & \infty & 2 & -3 & \infty \\ 0 & 2 & 0 & 4 & -1 & -8 \\ -4 & \infty & \infty & 0 & -5 & \infty \\ 5 & 7 & \infty & 9 & 0 & \infty \\ 3 & 5 & 10 & 7 & 2 & 0 \end{bmatrix}$$

$$A^5 = \begin{bmatrix} 0 & 6 & \infty & 8 & -1 & \infty \\ -2 & 0 & \infty & 2 & -3 & \infty \\ 0 & 2 & 0 & 4 & -1 & -8 \\ -4 & 2 & \infty & 0 & -5 & \infty \\ 5 & 7 & \infty & 9 & 0 & \infty \\ 3 & 5 & 10 & 7 & 2 & 0 \end{bmatrix}$$

$$A^6 = \begin{bmatrix} 0 & 6 & \infty & 8 & -1 & \infty \\ -2 & 0 & \infty & 2 & -3 & \infty \\ -5 & -3 & 0 & -1 & -6 & -8 \\ -4 & 2 & \infty & 0 & -5 & \infty \\ 5 & 7 & \infty & 9 & 0 & \infty \\ 3 & 5 & 10 & 7 & 2 & 0 \end{bmatrix}$$

final

$$A^0 = \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

$$A^1 = \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

$$A^2 = \begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

$$A^3 = \begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

$$A^4 = \begin{bmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix}$$

$$A^5 = \begin{bmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix}$$

final



KAGHAZ
www.kag hazpk

Q.5) a)

Action	visit order	stack (left $\rightarrow$ right, top at right)
1) discover B	B	[B]
2) discover C	B, C	[B, C]
3) discover F	B, C, F	[B, C, F]
4) discover H	B, C, F, H	[B, C, F, H]
5) " G	B, C, F, H, G	[B, C, F, H, G]
6) " D	B, C, F, H, G, D	[B, C, F, H, G, D]
7) pop D	-	[B, C, F, H, G]
8) pop G	-	[B, C, F, H]
9) discover I	B, C, F, H, G, D, I	[B, C, F, H, I]
10) pop I	-	[B, C, F, H]
11) " H	-	[B, C, F]
12) " F	-	[B, C]
13) " C	-	[B]
14) discover E	B, C, F, H, G, D, I, E	[B, E]
15) discover A	B, C, F, H, G, D, I, E, A	[B, E, A]
16) pop A	-	[B, E]
17) pop E	-	[B]
18) pop B	-	[]

order: B C F H G D I E A

* tree edges:	* back edges	* front edges	* cross-edges
$\hookrightarrow B \rightarrow C$	$\hookrightarrow A \rightarrow B$	$\hookrightarrow F \rightarrow I$	$\hookrightarrow A \rightarrow D$
$\hookrightarrow B \rightarrow E$	$\hookrightarrow D \rightarrow H$		$\hookrightarrow E \rightarrow H$
$\hookrightarrow C \rightarrow F$	$\hookrightarrow I \rightarrow H$		
$\hookrightarrow F \rightarrow H$			
$\hookrightarrow H \rightarrow G$			
$\hookrightarrow H \rightarrow I$			
$\hookrightarrow G \rightarrow D$			
$\hookrightarrow E \rightarrow A$			

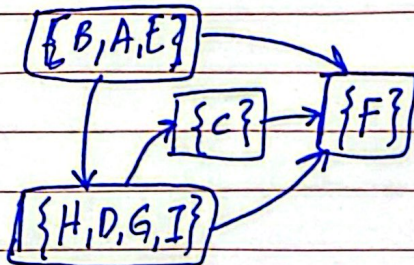
KAGHAZ
www.kaghaz.pk

c) strongly connected components

$$\hookrightarrow S_0 = \{B, A, E\} \quad \hookrightarrow S_1 = \{C\}$$

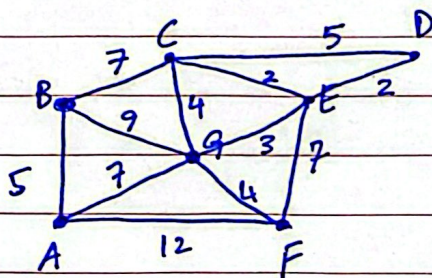
$$\hookrightarrow S_2 = \{F\}$$

$$\hookrightarrow S_3 = \{H, D, G, I\}$$



Q.6)

a)

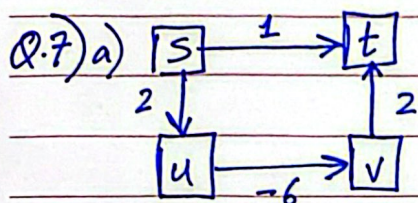


step	picked edge	S (after pick)	cut edges ^(next choices)	cumulative weight
0	—	{A}	A-F(12), A-M(7), A-B(5)	0
1	A-F(12)	{A, F}	A-M(7), A-B(5), F-E(7), F-M(7)	12
2	F-E(7)	{A, F, E}	A-M(7), E-M(3), E-D(2), A-B(5)	19
3	A-M(7)	{A, F, E, M}	M-B(9), M-C(4), M-F(4), A-B(5)	26
4	M-B(9)	{A, F, E, M, B}	B-C(7), M-C(4), A-B(5)	35
5	B-C(7)	{A, F, E, M, B, C}	C-D(5), C-E(2), M-C(4)	42
6	C-D(5)	{A, F, E, M, B, C, D}	—	47

cost = 47

KAGHAZ
www.kag haz.pk

b) Yes, both can produce different maximum spanning trees (with different sets of edges), but the total weight will remain the same (both are optimal)



↳ According to Dijkstra algorithm ^{would be 1, as} ~~should be 1~~ ^{instantly} from $s \rightarrow t = 1$. As that's the shortest edge.

But when other edges are traversed, the cost will become negative weighted

$$s \rightarrow u \rightarrow v \rightarrow t = -2$$

	s	t	u	v
s		(1, s)	(2, s)	(∞ , -)
st			(2, s)	(∞ , -)
stu				(-4, u)

b) No, adding a constant k , to all edges is not valid as adding k changes the cost of a path ($n(\text{edges}) \times k$), so paths w/ different lengths are distorted differently. So a path that was originally shorter, would become longer after addition of k . Hence the shortest-path ordering is not preserved.

c) Dijkstra works correctly only when negative edges leave the source node s , as all negative relaxations happen immediately during initialization (distance updates from s). After ~~at~~ that, all remaining edges are non-negative, so the standard correctness proof of Dijkstra applies.

